CSE 4553
Machine Learning
Lecture 9: Recurrent Neural Network (RNN)

Winter 2024

Dr. Hasan Mahmud | hasan@iut-dhaka.edu

Different types of data in machine learning

- **Spatial data:** Spatial data is actually tabular data, but its observation has spatial attributes. Spatial data is directly or indirectly references a specific geographical area or location.
- **Temporal data:** Temporal data is simply data that represents a state over time. Temporal data is collected to analyze weather patterns and other environmental variables, monitor traffic conditions, study demographic trends, and so on.
- **Time-series data:** is a sequence of data points collected over time intervals, allowing us to *track changes over time*. Time-series data can track changes over milliseconds, days, or even years. Time-series data is also sequential data
- **Sequential Data:** sequential Data refers to any data that contain elements that are ordered into sequences. Examples include time series, DNA sequences (see biomedical informatics) and sequences of user actions.

Real-life Sequence Learning Applications

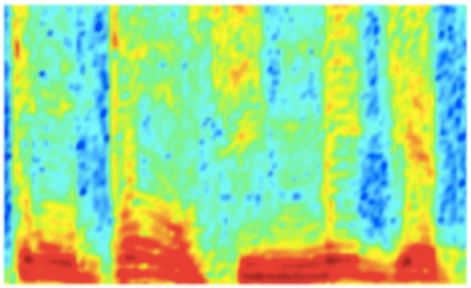
- RNNs can be applied to various type of sequential data to learn the temporal patterns.
  - Time-series data (e.g., stock price) → Prediction, regression
  - Raw sensor data (e.g., signal, voice, handwriting) → Labels or text sequences
  - Text → Label (e.g., sentiment) or text sequence (e.g., translation, summary, answer)
  - Image and video → Text description (e.g., captions, scene interpretation)

Task	Input	Output
Activity Recognition (Zhu et al. 2018)	Sensor Signals	Activity Labels
Machine translation (Sutskever et al. 2014)	English text	French text
Question answering (Bordes et al. 2014)	Question	Answer
Speech recognition (Graves et al. 2013)	Voice	Text
Handwriting prediction (Graves 2013)	Handwriting	Text
Opinion mining (Irsoy et al. 2014)	Text	Opinion expression

Sequence Sources

* Elements of a sequence occur in a certain order
* Elements depend on each other

AUDIO



Audio Spectrogram

IMAGES

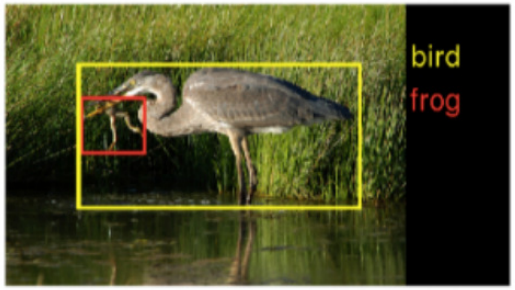


Image pixels

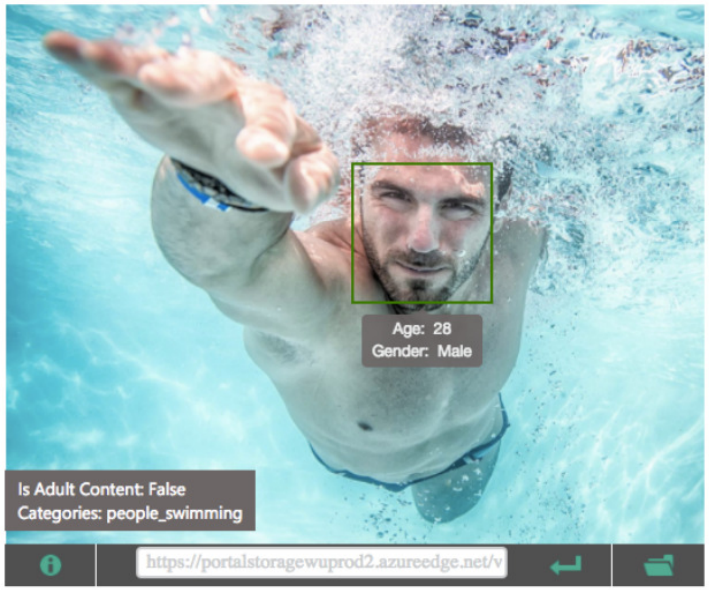
TEXT

0	0	0	0.2	0	0.7	0	0	0	...	...

Word, context, or document vectors

Sequence Applications: One-to-Many

- **Input**: fixed-size
- **Output**: sequence
- e.g., image captioning



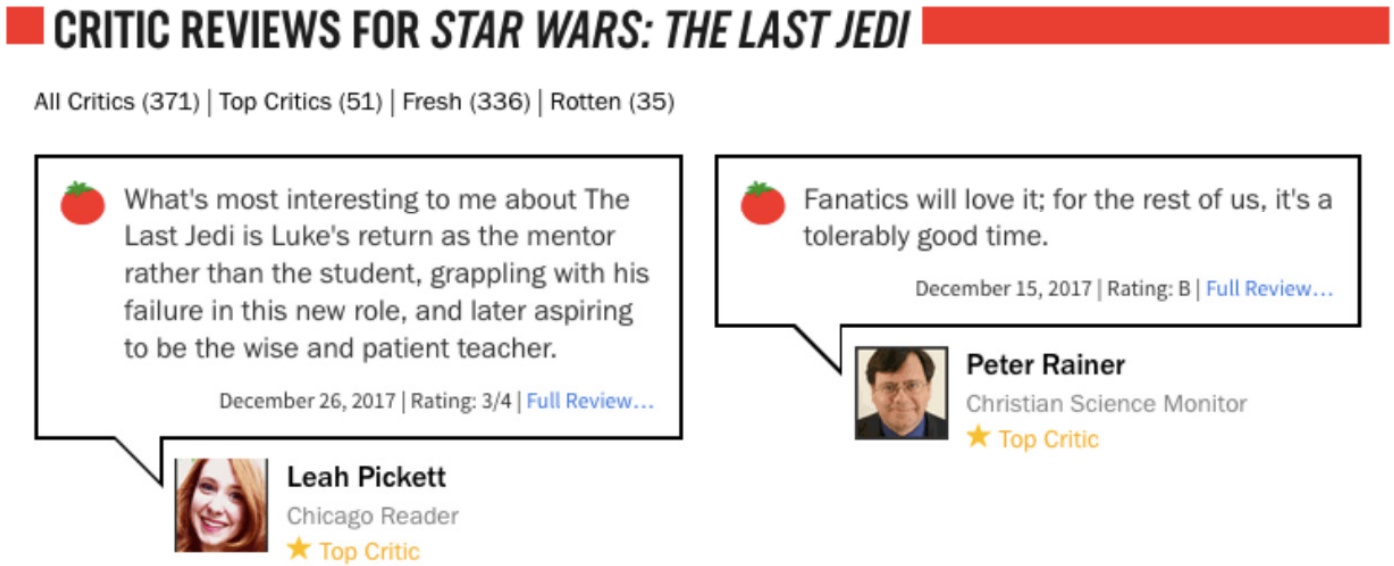
Features:

Feature Name	Value
Description	{ "type": 0, "captions": [{ "text": "a man swimming in a pool of water", "confidence": 0.7850108693093019 }] }
Tags	[{ "name": "water", "confidence": 0.9996442794799805 }, { "name": "sport", "confidence": 0.9504992365837097 }, { "name": "swimming", "confidence": 0.9062818288803101, "hint": "sport" }, { "name": "pool", "confidence": 0.8787588477134705 }, { "name": "water sport", "confidence": 0.631849467754364, "hint": "sport" }]
Image Format	jpeg
Image Dimensions	1500 x 1155
Clip Art Type	0 Non-clipart
Line Drawing Type	0 Non-LineDrawing
Black & White Image	False

Captions: https://www.microsoft.com/cognitive-services/en-us/computer-vision-api

Sequence Applications: Many-to-One

- **Input**: sequence
- **Output**: fixed-size
- e.g., sentiment analysis (hate? love?, etc)



https://www.rottentomatoes.com/m/star_wars_the_last_jedi

Sequence Applications: Many-to-Many

- **Input**: sequence
- **Output**: sequence
- e.g., language translation

1-of-N encoding

How to represent each word as a vector?

1-of-N Encoding lexicon = {apple, bag, cat, dog, elephant}

The vector is lexicon size.

Each dimension corresponds to a word in the lexicon

The dimension for the word is 1, and others are 0

apple = [1 0 0 0 0]

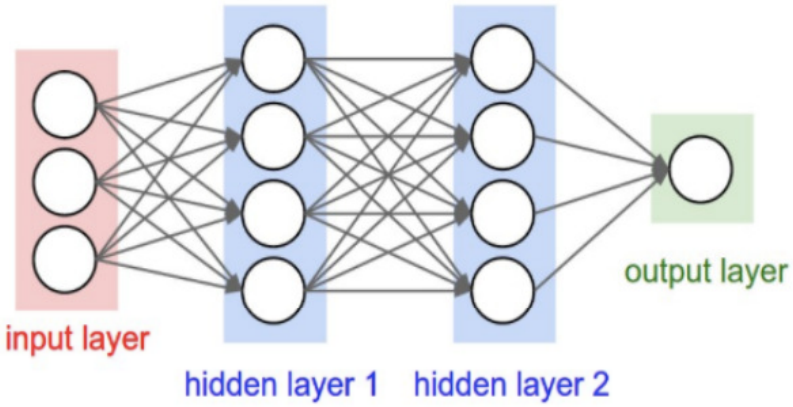
bag = [0 1 0 0 0]

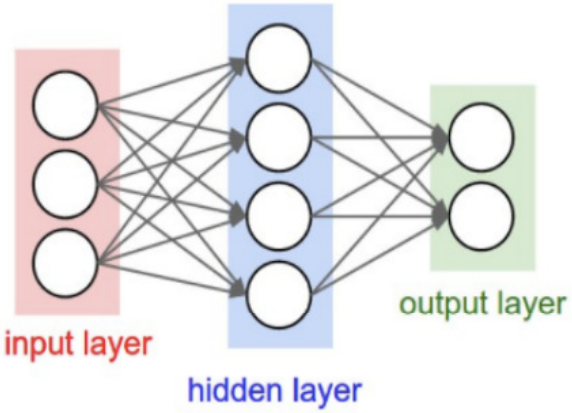
cat = [0 0 1 0 0]

dog = [0 0 0 1 0]

elephant = [0 0 0 0 1]

Recall: Feedforward Neural Networks





Problem: many model parameters!

Problem: no memory of past since weights learned independently

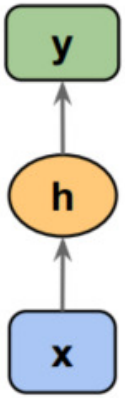
Each layer serves as input to the next layer with no loops

Figure Source: http://cs231n.github.io/neural-networks-1/

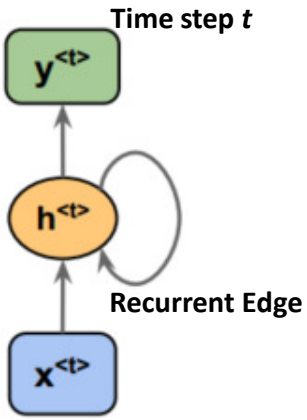
Recurrent Neural Networks

- Human brain deals with information streams. Most data is obtained, processed, and generated sequentially.
  - E.g., listening: soundwaves → vocabularies/sentences
  - E.g., action: brain signals/instructions → sequential muscle movements
- Human thoughts have persistence; humans don't start their thinking from scratch every second.
  - As you read this sentence, you understand each word based on your prior knowledge.
- The applications of standard Artificial Neural Networks (and also Convolutional Networks) are limited due to:
  - They only accepted a fixed-size vector as input (e.g., an image) and produce a fixed-size vector as output (e.g., probabilities of different classes).
  - These models use a fixed amount of computational steps (e.g. the number of layers in the model).
- Recurrent Neural Networks (RNNs) are a family of neural networks introduced to **learn sequential data**.
  - Inspired by the temporal-dependent and persistent human thoughts

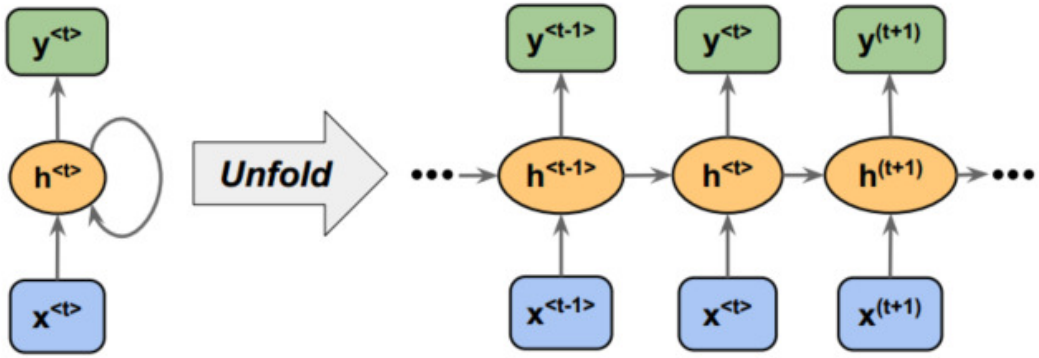
Recurrent Neural Networks



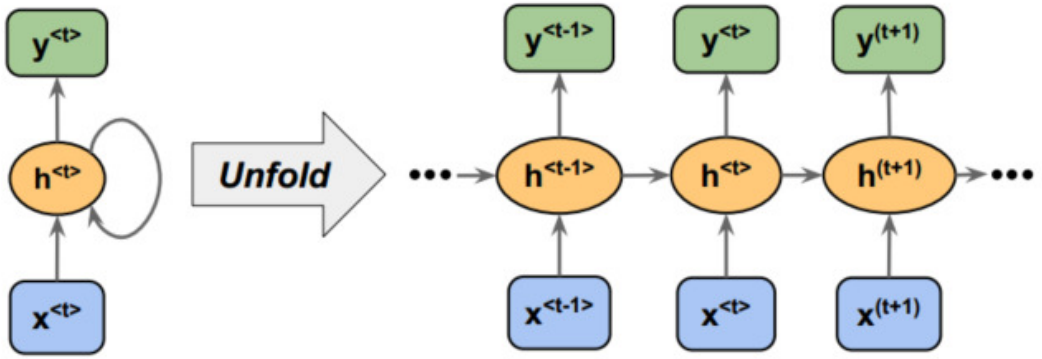
Feedforward Neural Network (FFNN)



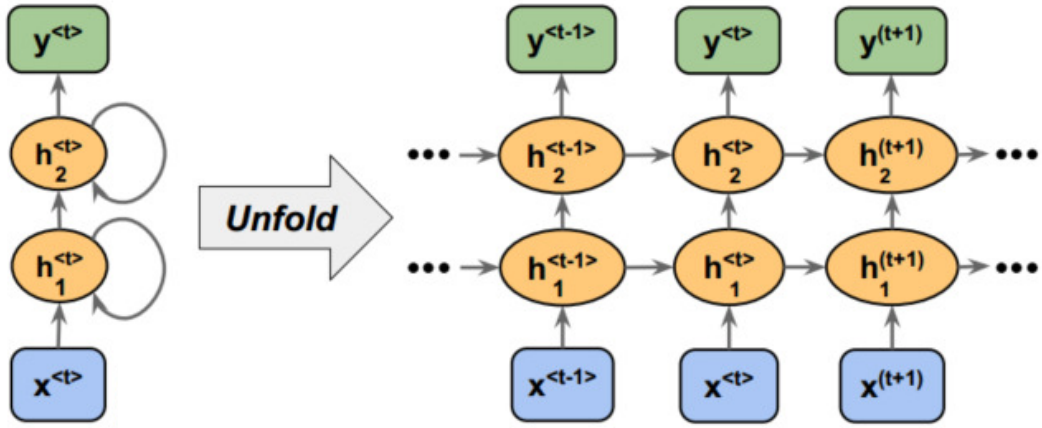
Recurrent Neural Network (RNN)



Single Layer RNN



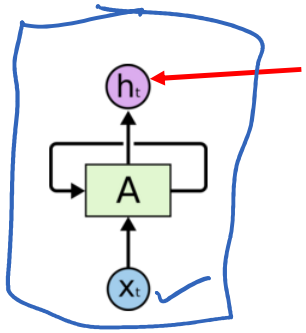
Single Layer RNN



Multilayer RNN

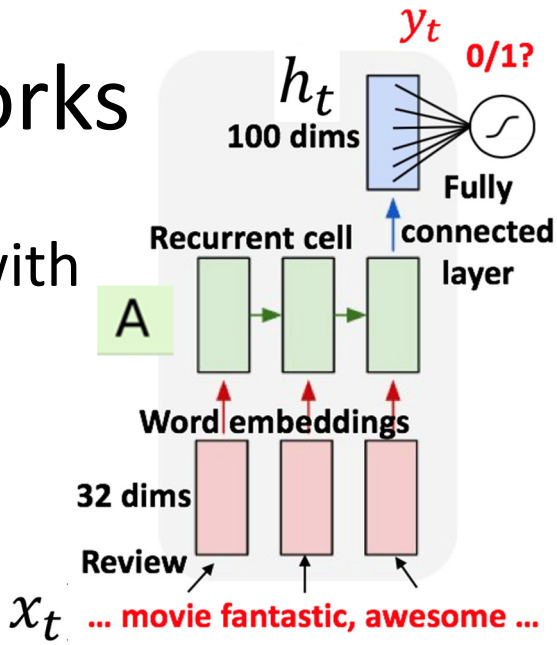
Recurrent Neural Networks

- Recurrent Neural Networks are networks with loops, allowing information to persist.



Recurrent Neural Networks have loops.

Output is to predict a vector h_t, where $output\ y_t = \varphi(h_t)$ at some time steps (t)



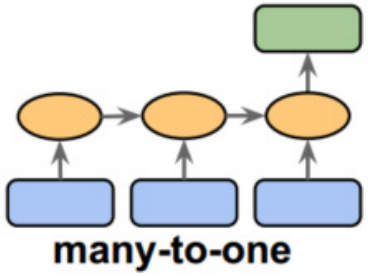
In the above diagram, a chunk of neural network, $A = f_W$, looks at some input x_t and outputs a value h_t. A loop allows information to be passed from one step of the network to the next.

new state — old state

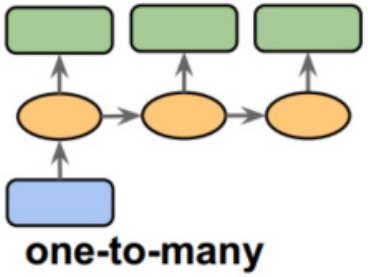
$$h_t = f_W(h_{t-1}, x_t)$$

function with parameter W — Input vector at some time step

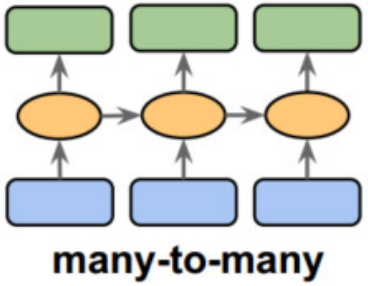
Types of sequence modelling in RNN



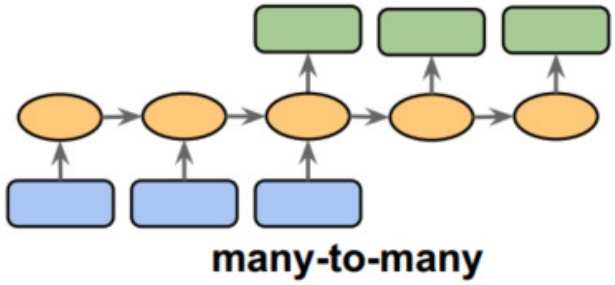
Input: Data in sequence
Output: Class label
Sentiment Classification



Input: Not sequence
Output: A sequence
Image Captioning



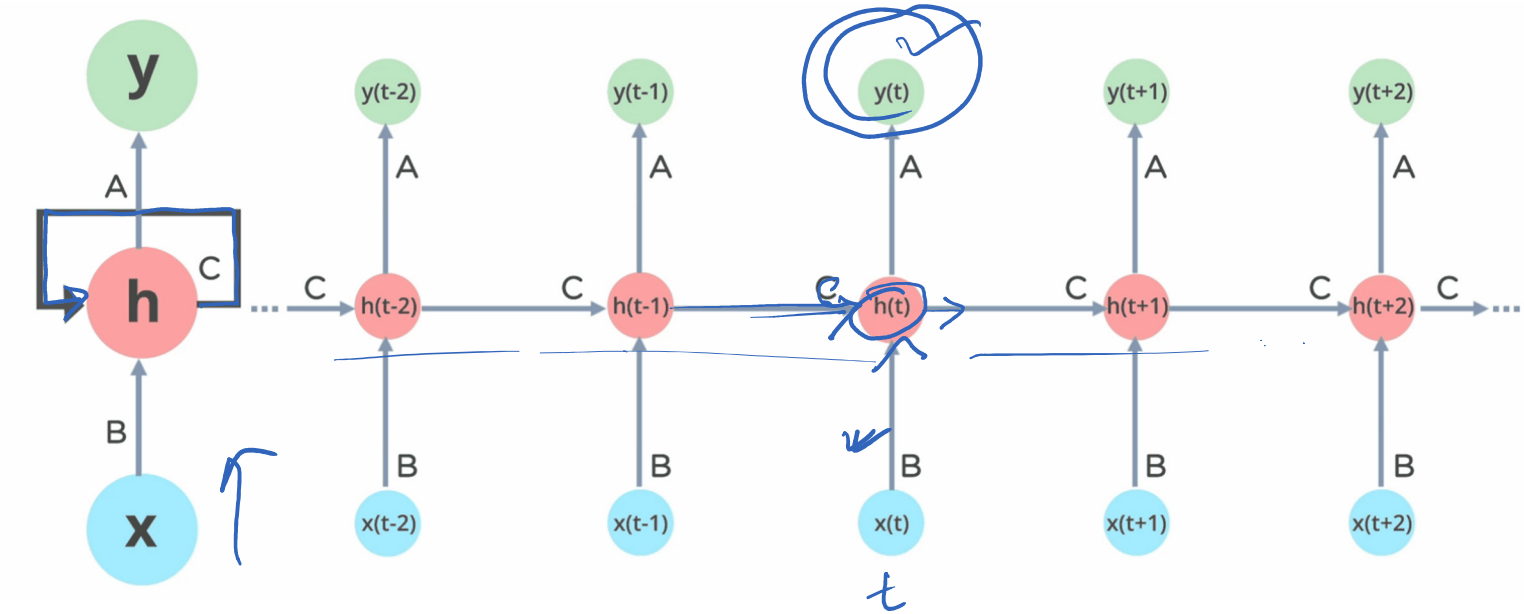
Input: A sequence
Output: A sequence
NER recognition



Input: A sequence
Output: A sequence
Language Translation

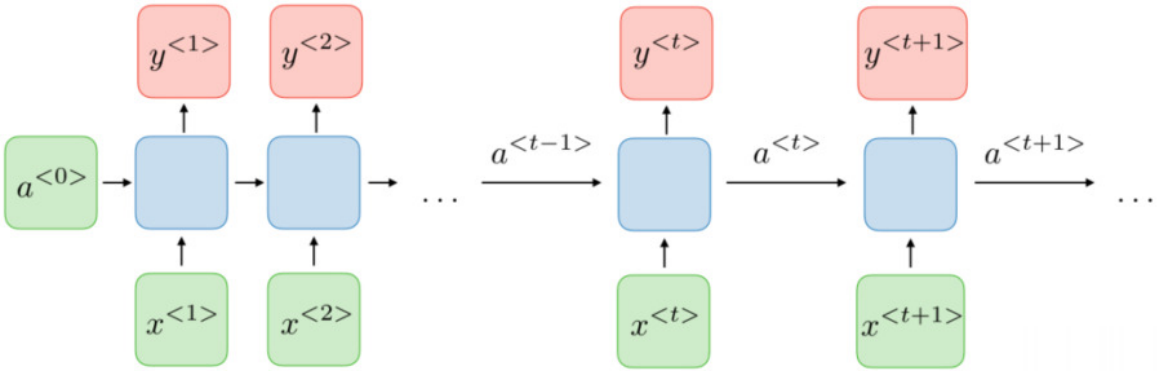
Image source: Sebastian Raschka, Vahid Mirjalili. *Python Machine Learning. 3rd Edition.* Packt, 2019

Unfolding RNN



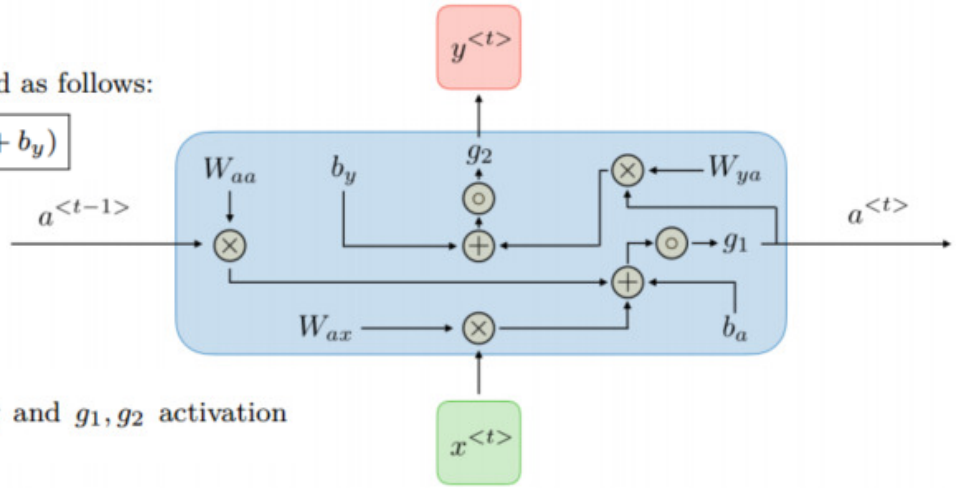
A recurrent neural network can be thought of as multiple copies of the same network, each passing a message to a successor. The diagram above shows what happens if we **unroll the loop**.

Unfolding RNN



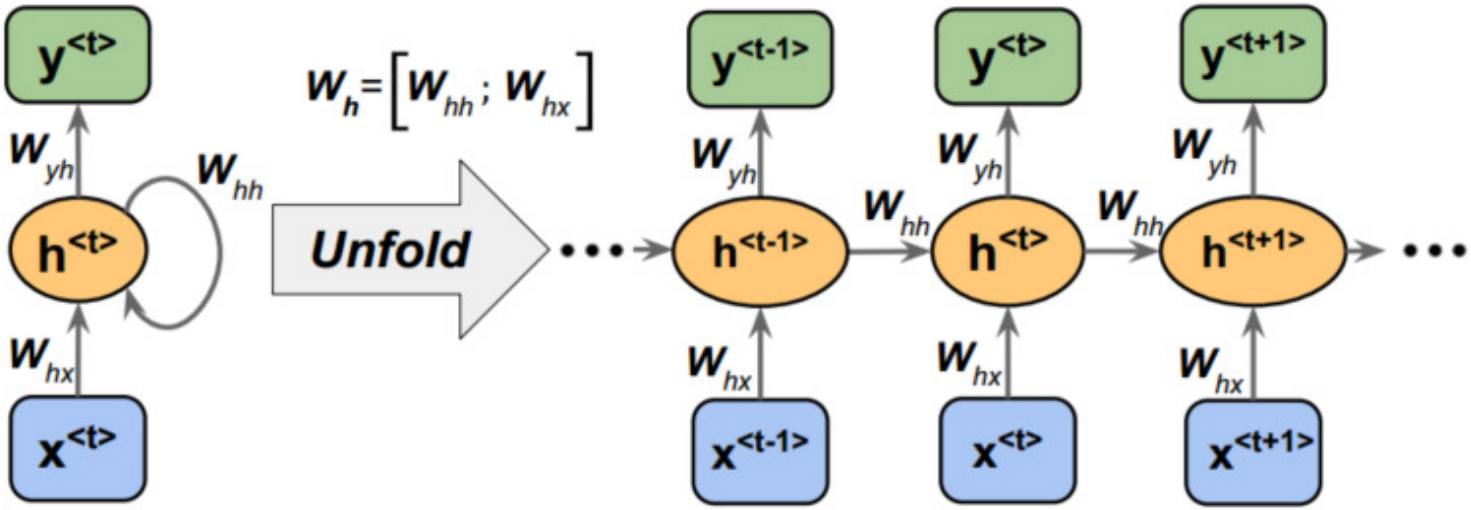
For each timestep t, the activation $a^{<t>}$ and the output $y^{<t>}$ are expressed as follows:

$$\boxed{a^{<t>} = g_1(W_{aa}a^{<t-1>} + W_{ax}x^{<t>} + b_a)} \quad \text{and} \quad \boxed{y^{<t>} = g_2(W_{ya}a^{<t>} + b_y)}$$



where $W_{ax}, W_{aa}, W_{ya}, b_a, b_y$ are coefficients that are shared temporally and g_1, g_2 activation functions

Weight Matrices of RNN



Forward propagation of RNN

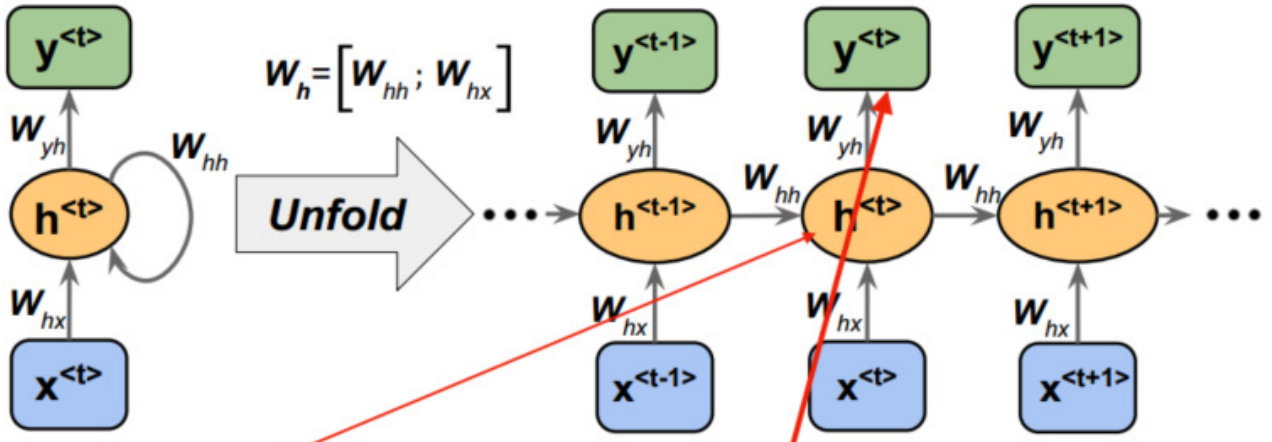


Image source: Sebastian Raschka, Vahid Mirjalili. *Python Machine Learning. 3rd Edition.* Packt, 2019

Net input:

$$\mathbf{z}_h^{\langle t \rangle} = \mathbf{W}_{hx}\mathbf{x}^{\langle t \rangle} + \mathbf{W}_{hh}\mathbf{h}^{\langle t-1 \rangle} + \mathbf{b}_h$$

Activation:

$$\mathbf{h}^{\langle t \rangle} = \sigma_h\left(\mathbf{z}_h^{\langle t \rangle}\right)$$

Net input:

$$\mathbf{z}_y^{\langle t \rangle} = \mathbf{W}_{yh}\mathbf{h}^{\langle t \rangle} + \mathbf{b}_y$$

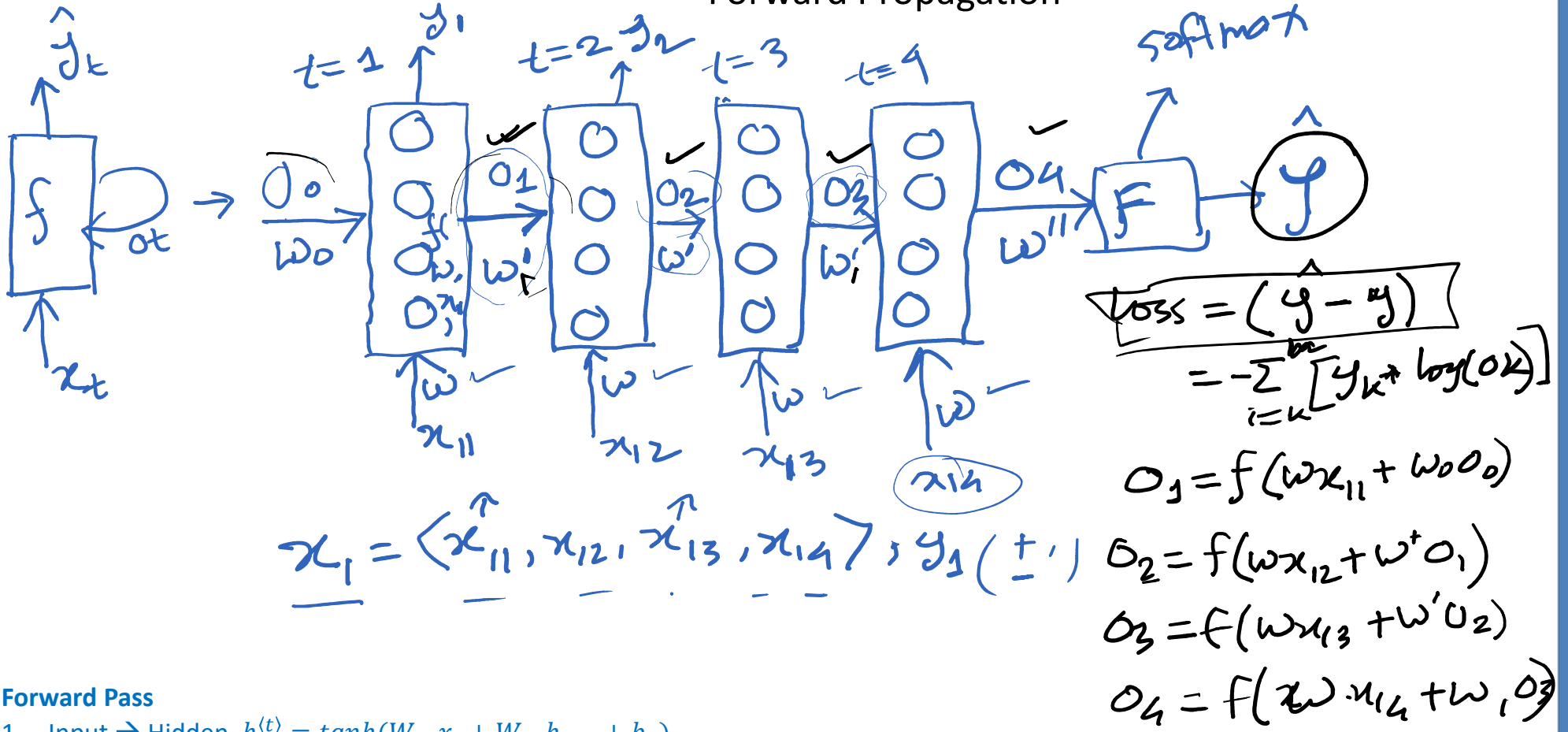
Output:

$$\mathbf{y}^{\langle t \rangle} = \sigma_y\left(\mathbf{z}_y^{\langle t \rangle}\right)$$

Forward Pass

1. Input → Hidden, $h^{\langle t \rangle} = tanh(W_{hx}x_t + W_{hh}h_{t-1} + b_h)$
2. Hidden → Output, $y^{\langle t \rangle} = softmax(W_{yh}h_t + b_y)$

Forward Propagation



Forward Pass

1. Input → Hidden, $h^{\langle t \rangle} = tanh(W_{hx}x_t + W_{hh}h_{t-1} + b_h)$
2. Hidden → Output, $y^{\langle t \rangle} = softmax(W_{yh}h_t + b_y)$

Backward propagation of RNN

The output at each time step t is given by:

$y^{\langle t \rangle} = softmax(W_{yh} h_t + b_y)$

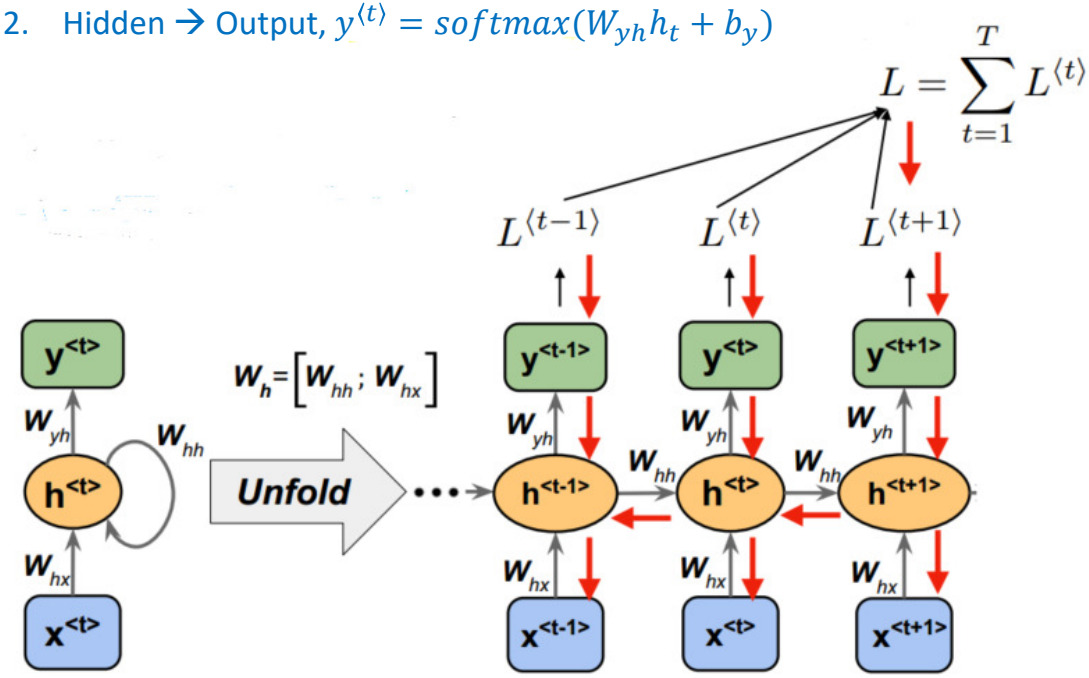
The loss L is the sum of the losses at each time step:

$$L = \sum_{t=1}^{T} L^{\langle t \rangle}$$

- Backward Propagation steps:
  - Compute gradient of loss with respect to output layer parameters, $\boldsymbol{W_{yh}, b_y}$
  - Compute gradient of loss with respect to hidden layer parameter, $\boldsymbol{W_{hx}, W_{hh}, b_h}$
  - Update weights using gradient descent

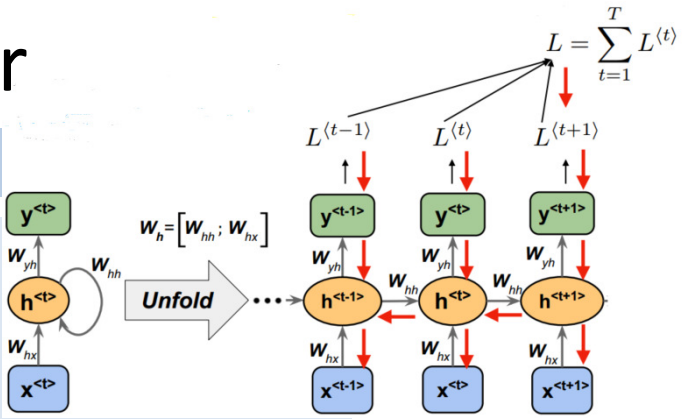
Forward Pass

1. Input → Hidden, $h^{\langle t \rangle} = tanh(W_{hx} x_t + W_{hh} h_{t-1} + b_h)$
2. Hidden → Output, $y^{\langle t \rangle} = softmax(W_{yh} h_t + b_y)$



Gradients for output layer

Gradients for output layer parameters (W_{yh}, b_y) with respect to loss



Gradient of Loss with respect to output $y^{\langle t \rangle}$ is:

$$\delta^{\langle t \rangle} = \frac{\partial L^{\langle t \rangle}}{\partial y^{\langle t \rangle}}$$

which depends on the chosen loss function. For cross-entropy with softmax, the loss/error is,

$$\delta^{\langle t \rangle} = y^{\langle t \rangle} - \hat{y}^{\langle t \rangle}$$

Gradient of Loss with respect to W_{yh}

$$\frac{\partial L}{\partial W_{yh}} = \sum_{t=1}^{T} \delta^{\langle t \rangle} \cdot (h^{\langle t \rangle})^T$$

Multiply the error by the hidden state's value at each step (loss multiplied by hidden state vale)

Gradient of Loss with respect to b_y

$$\frac{\partial L}{\partial b_y} = \sum_{t=1}^{T} \delta^{\langle t \rangle}$$

Gradients for Hidden layer

Gradients for hidden layer parameters ($\boldsymbol{W_{hx}, W_{hh}, b_h}$) with respect to loss

Gradient of Loss with respect to hidden state $h^{\langle t \rangle}$, the error comes from two places:

1. The immediate contribution from the output at time t: $(\mathrm{W_{yh}})^T \frac{\partial L}{\partial y^{\langle t \rangle}}$
2. The contribution from the future hidden state $h^{\langle t+1 \rangle}$ (because $h^{\langle t \rangle}$ also affects $h^{\langle t+1 \rangle}$ through (W_{hh}):

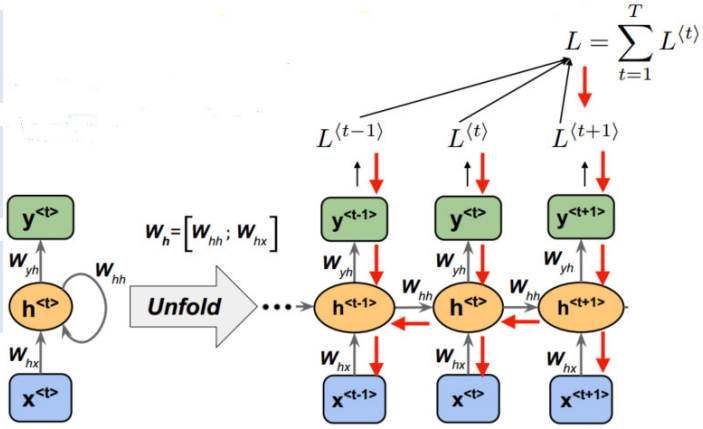
$$\frac{\boldsymbol{\partial L}}{\boldsymbol{\partial h^{\langle t \rangle}}} = (\boldsymbol{W_{yh}})^{\boldsymbol{T}} \frac{\boldsymbol{\partial L}}{\boldsymbol{\partial y^{\langle t \rangle}}} + (\boldsymbol{W_{\mathrm{hh}}})^{\boldsymbol{T}} \frac{\boldsymbol{\partial L}}{\boldsymbol{\partial h^{\langle t+1 \rangle}}} \cdot \boldsymbol{f'(z^{\langle t+1 \rangle})}$$

We define, $z^{\langle t \rangle} = \mathrm{W_{hx}} x^{\langle t \rangle} + \mathrm{W_{hh}} h^{\langle t-1 \rangle} + b^h$, $h^{\langle t \rangle} = f\left(z^{\langle t \rangle}\right)$, f' is the derivative of hidden activation

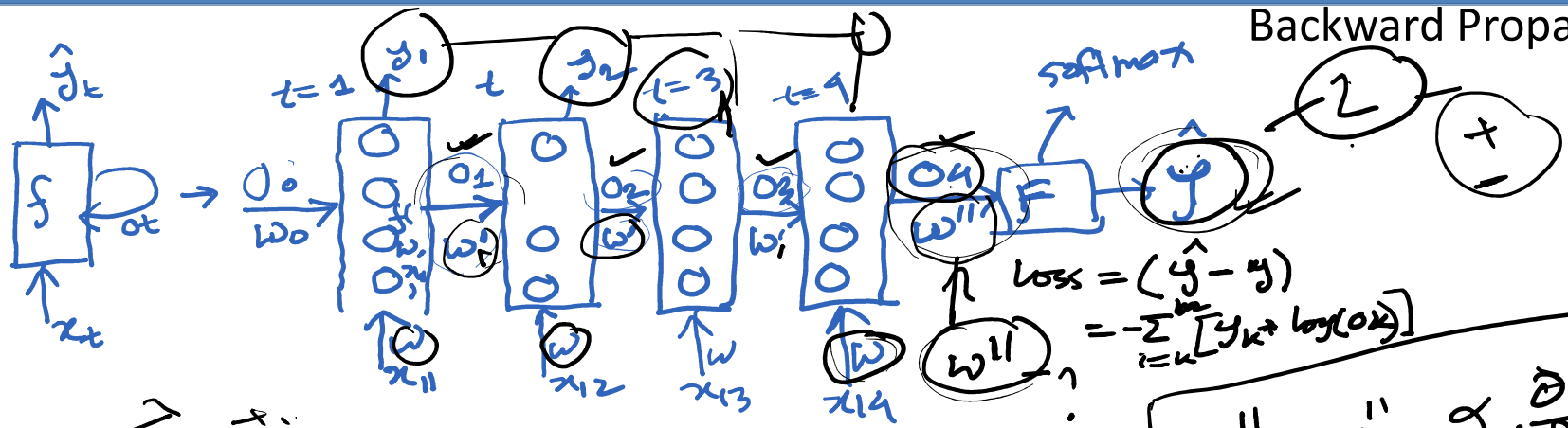
Gradient of $\boldsymbol{W_{\mathrm{hh}}}$: $\frac{\boldsymbol{\partial L}}{\boldsymbol{\partial W_{\mathrm{hh}}}} = \sum_{\boldsymbol{t=1}}^{\boldsymbol{T}} [\frac{\boldsymbol{\partial L}}{\boldsymbol{\partial h^{\langle t \rangle}}} \cdot \boldsymbol{f' z^{\langle t \rangle}}] \cdot (\boldsymbol{h^{\langle t-1 \rangle}})^{\boldsymbol{T}}$

Gradient of $\boldsymbol{W_{\mathrm{hx}}}$: $\frac{\boldsymbol{\partial L}}{\boldsymbol{\partial W_{\mathrm{hx}}}} = \sum_{\boldsymbol{t=1}}^{\boldsymbol{T}} [\frac{\boldsymbol{\partial L}}{\boldsymbol{\partial h^{\langle t \rangle}}} \cdot \boldsymbol{f' z^{\langle t \rangle}}] \cdot (\boldsymbol{x^{\langle t \rangle}})^{\boldsymbol{T}}$

Gradient of $\boldsymbol{b_{\mathrm{h}}}$: $\frac{\boldsymbol{\partial L}}{\boldsymbol{\partial b_{\mathrm{h}}}} = \sum_{\boldsymbol{t=1}}^{\boldsymbol{T}} [\frac{\boldsymbol{\partial L}}{\boldsymbol{\partial h^{\langle t \rangle}}} \cdot \boldsymbol{f' z^{\langle t \rangle}}]$



Backward Propagation



$$\frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial L}{\partial w^{11}} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial w^{11}}$$

$$w^{11} = w^{11} - \alpha \cdot \frac{\partial L}{\partial w^{11}}$$

$$w = w - \alpha \cdot \frac{\partial L}{\partial w}$$

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial o_4} \cdot \frac{\partial o_4}{\partial w}$$

→ vanishing gradient

0.000000 0001

→ Exploding grad.

→ LSTM

Two problems with RNN

- Vanishing gradient
- Exploding gradient